

Two Pointers & Sliding Window

$O(n^2)$ ని $O(n)$ చేసే అందమైన ఉపాయం. దానికి ఒక చెప్పని షరతు ఉంది, మరియు ఆ షరతు విరిగినప్పుడు code **crash** కాదు – తప్పు జవాబు ఇస్తుంది.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – $n = 20,000$ దగ్గర brute force **3,879** రెట్లు నెమ్మది. రెండేది – array lo ఒక్క ఋణాత్మక సంఖ్య ఉంటే sliding window **14.77% cases lo తప్పు**. మూడేది – "సరిగ్గా K" అని నేరుగా లెక్కపెడితే **47.8% తప్పు**, మరియు సరైన జవాబు ఒక తేడా.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

DSA DEEP 01 యొక్క అదే ఆకారం, వేరే ALGORITHM మద

Binary search కి ఒక చెప్పని షరతు ఉంది: **predicate ఏకదిశగా ఉండాలి**. అది విరిగితే జవాబు మౌనంగా తప్పు.

Sliding window కి కూడా అదే ఆకారపు షరతు ఉంది: **కిటికీ పెరిగితే కొలత ఏకదిశగా మారాలి**. ధనాత్మక సంఖ్యలతో sum ఎప్పుడూ పెరుగుతుంది – కాబట్టి "ఎక్కువైంది, ఎడమవైపు కుంచించు" అనేది అర్థవంతం.

ఒక్క ఋణాత్మక సంఖ్య ఆ హామీని తుడిచేస్తుంది. §6 lo కొలుస్తాం: **0% → 14.77%** తప్పు జవాబులు.

ఈ రెండు algorithms నీ ఒకే ప్రశ్న కాపాడుతుంది: "నేను ఈ దిశలో కదిలితే, వెనక్కి రావాల్సిన అవసరం ఎప్పటికీ ఉండదని నాకెలా తెలుసు?"

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. ఈ pattern ఏమి చేస్తుంది – మరియు ఎందుకు పనిచేస్తుంది
2. అడగాల్సిన ప్రశ్నలు – ఏ హామీ మీద ఆధారపడుతున్నాం?

Part 2 – మొదటి విరుపు: $O(n^2)$ గోడ

3. Step – అన్ని subarrays చూడటం
4. మొదటి విరుపు – $n = 20,000$ దగ్గర 3,879 రెట్లు

Part 3 – రెండో విరుపు: ఒక్క ఋణాత్మక సంఖ్య

5. Step – కిటికీ పెంచు, ఎక్కువైతే కుంచించు
6. రెండో విరుపు – ఋణాత్మకాలతో 14.77% తప్పు
7. Step – షరతుని స్పష్టంగా రాయడం

Part 4 – మూడో విరుపు: "సరిగ్గా K"

8. Step – "గరిష్టంగా K" లెక్కపెట్టడం
9. మూడో విరుపు – "సరిగ్గా K" ని నేరుగా లెక్కపెడితే 47.8% తప్పు
10. Step – తేడాగా రాయడం

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code · 1.2 లక్షల cases · mutation testing
12. ఒకే bug, రెండు వేర్వేరు ఫలితాలు
13. Interview lo ఎలా రాయాలి

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. ఈ pattern ఏమి చేస్తుంది – మరియు ఎందుకు పనిచేస్తుంది

ఒక array lo అన్ని subarrays $n(n+1)/2$ ఉంటాయి. $n = 1$ లక్ష అయితే అది 500 కోట్లు. వాటన్నిటినీ చూడటం సాధ్యం కాదు.

Sliding window యొక్క వాదన ఇది:

కుడి pointer r ప్రతి స్థానానికీ ఒకసారి వెళ్తుంది. ఎడమ pointer l కూడా ఒకసారి. కాబట్టి మొత్తం పని $2n$ – కానీ మనం $n(n+1)/2$ subarrays అన్నిటినీ పరిగణించినట్టే.

ఆ వాదన నిజం కావాలంటే ఒక్క విషయం నిజం కావాలి: l ఎప్పుడూ వెనక్కి వెళ్ళకూడదు.

మరియు అదే చెప్పని షరతు

' l ' వెనక్కి వెళ్ళనవసరం లేదు అని ఎందుకు? ఎందుకంటే ' r ' కుడివైపుకి కదిలినప్పుడు కిటికీలోని sum పెరుగుతుంది మాత్రమే. కాబట్టి ఇప్పుడు చెల్లని ఒక ఎడమ స్థానం, భవిష్యత్తులో కూడా చెల్లదు.

ఆ "పెరుగుతుంది మాత్రమే" అనేది ధనాత్మక సంఖ్యల వల్ల వస్తుంది. అది ఒక ఊహ కాదు – అది ఒక ఆధారం.

§6 lo ఆ ఆధారాన్ని తీసేసి ఏమవుతుందో కొలుస్తాం.

2. అడగాల్సిన ప్రశ్నలు – ఏ హామీ మీద ఆధారపడుతున్నాం?

ప్రశ్న	ఎందుకు అడుగుతున్నాం
సంఖ్యలు ఋణాత్మకం కావచ్చా?	§6. ఇదే ఏకైక అత్యంత ముఖ్యమైన ప్రశ్న
సున్నా ఉండొచ్చా?	సున్నా సరే – sum తగ్గదు కాబట్టి ఏకదిశత నిలుస్తుంది (§6 lo కొలిచాను)
Subarray కావాలా, subsequence కావాలా?	Subsequence అయితే sliding window పనికిరాదు – అది వేరే problem
పొడవైనది కావాలా, ఎన్ని అని కావాలా?	§12 – ఒకే bug ఒక ప్రశ్నలో హానికరం కాదు, ఇంకొక దాంట్లో ఘోరం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
"సరిగ్గా K" అనే మాట ఉందా?	ఉంటే §9 – నేరుగా లెక్కపెట్టడం పనిచేయదు
కిటికీ పొడవు స్థిరమా?	స్థిరం అయితే ఇది ఇంకా సులభం – కుంచిం చే loop అవసరం లేదు

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా రాశారని తెలుస్తుంది

"ఈ array lo ఋణాత్మక సంఖ్యలు ఉండొచ్చా?"

ఇది ఒక సాధారణ clarifying ప్రశ్నలా అనిపిస్తుంది. నిజానికి అది మొత్తం విధానాన్నే నిర్ణయిస్తుంది:

అన్నీ ≥ 0 → sliding window, $O(n)$.

ఋణాత్మకాలు ఉన్నాయి → sliding window పూర్తిగా తప్పు. Prefix sums + ఒక sorted నిర్మాణం కావాలి, $O(n \log n)$ (§14).

మరియు interviewer ఈ ప్రశ్న అడగకపోతే – అడగడమే మే పని.

Part 2 – మొదటి విరుపు: $O(n^2)$ గోడ

3. Step – అన్ని subarrays చూడటం

```
const brute = (a, k) => { let best = 0;
  for (let i = 0; i < a.length; i++) { let s = 0;
    for (let j = i; j < a.length; j++) { s += a[j]; if (s <= k) best = Math.max(best, j-i+1); } }
  return best; };
```

ఇది **ఖచ్చితంగా సరైనది**, ఎప్పుడూ, ఏ input కైనా – ఋణాత్మకాలతో సహా. అందుకే §11 lo ఇదే మన నిజం (reference).

4. మొదటి విరుపు – $n = 20,000$ దగ్గర 3,879 రెట్లు

"sum ≤ k ఉన్న పొడవైన subarray" · ధనాత్మక సంఖ్యలు · 3 runs lo ఉత్తమమైనది

n	brute force $O(n^2)$	రెండు pointers $O(n)$	ఎన్ని రెట్లు
100	0.1 ms	0.01 ms	22×
1,000	0.4 ms	0.04 ms	11×
5,000	10.3 ms	0.02 ms	452×
20,000	157.6 ms	0.04 ms	3879×
1,00,000	(చాలా నెమ్మది)	0.30 ms	–

N ని 4 రెట్లు పెంచితే (5,000 → 20,000) BRUTE FORCE 15 రెట్లు నెమ్మది

అదే $O(n^2)$ యొక్క అర్థం – n రెట్టింపు అయితే పని నాలుగు రెట్లు.

Sliding window ఆ మూడు వరుసల్లోనూ **0.02-0.04 ms** దగ్గరే ఉంది.

n = 1 లక్ష దగ్గర brute force ~ 10^{10} ఆపరేషన్లు – **గంటల కొద్దీ**. Sliding window: **0.30 ms**.

కానీ DSA Deep 01 యొక్క పాఠం ఇక్కడా వర్తిస్తుంది – **మొదటి రెండు వరుసలు చూడండి**. n = 1,000 దగ్గర లాభం 22× నుంచి **11×** కి తగ్గింది. చిన్న n దగ్గర స్థిరాంకాలు మరియు cache ఆధిపత్యం చెలాయిస్తాయి, మరియు Big-O ఇంకా మాట్లాడటం మొదలుపెట్టలేదు.

Part 3 – రెండో విరుపు: ఒక్క ఋణాత్మక సంఖ్య

5. Step – కిటికీ పెంచు, ఎక్కువైతే కుంచించు

```
function longestSumAtMost(a, k) {
  let best = 0, s = 0, l = 0;
  for (let r = 0; r < a.length; r++) {
    s += a[r]; // కుడివైపు పెంచు
    while (s > k && l <= r) s -= a[l++]; // ఎక్కువైంది → ఎడమవైపు కుంచించు
    if (s <= k) best = Math.max(best, r - l + 1);
  }
  return best;
}
```

ఆరు పంక్తులు. `l` మరియు `r` రెండూ ముందుకే కదులుతాయి, కాబట్టి $O(n)$.

ఇప్పుడు దీన్ని brute force తో పోల్చుదాం – అన్ని రకాల arrays మీద.

6. రెండో విరుపు – ఋణాత్మకాలతో 14.77% తప్పు

2,00,000 యాదృచ్ఛిక cases • sliding window vs brute force

array lo సంఖ్యలు	తప్పు జవాబులు	%
అన్నీ ధనాత్మకం (1..9)	0	0.00%
సున్నా కూడా (0..9)	0	0.00%
ఒక ఋణాత్మకం (-1..9)	3,541	1.77%
ఋణాత్మకాలు (-5..9)	29,547	14.77%

సున్నా సరే -1 ఒక్కటి చాలు

సున్నాలతో 0 తప్పులు – ఎందుకంటే సున్నా కలిపినా sum తగ్గదు. ఏకదిశత నిలిచింది.

కనిష్ట విలువ -1 కి పడగానే - 3,541 తప్పు జవాబులు. అదే ఒక్క మార్పు.

ఇది పరిమాణం గురించి కాదు; ఇది దిశ గురించి. ఒక్క ఋణాత్మక సంఖ్య "sum ఎప్పుడూ పెరుగుతుంది" అనే హామీని పూర్తిగా రద్దు చేస్తుంది.

ఒక నిర్దిష్ట ఉదాహరణ, ఐదు మూలకాలు మాత్రమే:

$a = [5, 8, -1, 1, -1]$ · $k = 7 \rightarrow$ window 3, నిజం 4

ఎందుకు విరిగింది - అడుగు అడుగునా

$r=1$ దగ్గర $\text{sum} = 13 > 7$. కాబట్టి window ఎడమవైపు కుంచిచి l ని ముందుకి తోస్తుంది.

కానీ నిజమైన జవాబు $[8, -1, 1, -1]$ - పొడవు 4, sum 7. అది $l = 1$ దగ్గర మొదలవుతుంది.

Window l ని 1 దాటి ముందుకి తోసేసింది, ఎందుకంటే ఆ క్షణంలో sum ఎక్కువగా ఉంది. తర్వాత వచ్చే ఋణాత్మక సంఖ్యలు ఆ sum ని తగ్గిస్తాయని దానికి తెలియదు - మరియు l వెనక్కి వెళ్ళలేదు.

"I వెనక్కి వెళ్ళనవసరం లేదు" అనే ఊహ $O(n)$ ని ఇచ్చింది. అదే ఊహ ఇక్కడ తప్పు జవాబు ఇచ్చింది.

7. Step – షరతుని స్పష్టంగా రాయడం

DSA Deep 01 lo binary search కి ఒక వాక్యం రాశాం. ఇక్కడ కూడా అదే:

SLIDING WINDOW రాసే ముందు ఈ వాక్యాన్ని పూర్తి చేయండి

"I ఇప్పుడు చెల్లదు అని తేలితే, r ఇంకా కుడివైపుకి వెళ్ళినా అది చెల్లదు - ఎందుకంటే ____."

$\text{sum} \leq k$, అన్నీ $\geq 0 \rightarrow$ "...ఎందుకంటే r కదిలితే sum పెరుగుతుంది మాత్రమే." ✓ ఒక నిరూపణ.

$\text{sum} \leq k$, ఋణాత్మకాలు ఉన్నాయి $\rightarrow$ "...ఎందుకంటే sum... తగ్గొచ్చు కూడా." ✗ వాక్యం పూర్తి కాలేదు.

గరిష్టంగా K వేర్వేరు మూలకాలు $\rightarrow$ "...ఎందుకంటే r కదిలితే వేర్వేరు మూలకాల సంఖ్య పెరుగుతుంది మాత్రమే." ✓

వాక్యం పూర్తి కాకపోతే ప్రత్యామ్నాయాలు:

పరిస్థితి	ఏమి వాడాలి
ఋణాత్మకాలు, $\text{sum} = k$ సరిగ్గా	Prefix sum + hash map, $O(n)$
ఋణాత్మకాలు, $\text{sum} \leq k$ పొడవైనది	Prefix sum + monotonic deque లేదా sorted నిర్మాణం, $O(n \log n)$
ఋణాత్మకాలు, sum గరిష్టం	Kadane's algorithm – పూర్తిగా వేరే ఆలోచన

Part 4 – మూడో విరుపు: "సరిగ్గా K"

8. Step – "గరిష్ఠంగా K" లెక్కపెట్టడం

ఇప్పుడు ఒక లెక్కపెట్టే ప్రశ్న: "గరిష్ఠంగా K వేర్వేరు మూలకాలు ఉన్న subarrays ఎన్ని?"

```
function atMostKDistinct(a, K) {  
  if (K < 0) return 0;  
  let c = 0, l = 0; const m = new Map();  
  for (let r = 0; r < a.length; r++) {  
    m.set(a[r], (m.get(a[r]) || 0) + 1);  
    while (m.size > K) { const x = a[l++]; const v = m.get(x) - 1;  
      if (v === 0) m.delete(x); else m.set(x, v); }  
    c += r - l + 1; // ఈ r తో ముగిసే *అన్ని* చెల్లుబాటు  
  }  
  return c;  
}
```

అ $c += r - l + 1$ అనే పంక్తి ఈ PATTERN యొక్క గుండె

$[l, r]$ చెల్లుబాటు అయితే, $[l+1, r]$, $[l+2, r]$... $[r, r]$ అన్నీ చెల్లుతాయి – అవి చిన్నవి, కాబట్టి వాటిలో వేర్వేరు మూలకాలు ఇంకా తక్కువ.

కాబట్టి ఒక్క అడుగులో $r - l + 1$ subarrays లెక్కలోకి వస్తాయి. ఒక్కొక్కటిగా కాదు.

§11 lo ఈ పంక్తిని $c += 1$ గా మార్చిస్తే 150 lo 150 విఫలం.

9. మూడో విరుపు – "సరిగ్గా K" ని నేరుగా లెక్కపెడితే 47.8% తప్పు

ఇప్పుడు "సరిగ్గా K వేర్వేరు". సహజమైన మార్పు: $c += r-l+1$ కి బదులు $\text{if (m.size === K) } c++$.

"సరిగ్గా K వేర్వేరు మూలకాలు" · 1,00,000 యాదృచ్ఛిక cases

పద్ధతి	తప్పు జవాబులు	%
సహజమైన window ($= K$ లెక్కపెట్టు)	47,824	47.8%
$\text{atMost}(K) - \text{atMost}(K-1)$	0	0.0%

ఉదా: $a = [4, 3, 3]$ · $K = 1 \rightarrow$ సహజమైనది 3, నిజం 4

మూడు మూలకాలు • జవాబు 3 కి బదులు 4

$a = [4, 3, 3]$, $K = 1$. సరిగ్గా ఒక వేర్వేరు మూలకం ఉన్న subarrays: $[4]$, $[3]$, $[3]$, $[3, 3]$ – నాలుగు.

సహజమైన window ప్రతి r దగ్గర ఒక్కటే లెక్కపెడుతుంది, ఎందుకంటే l ఒకే ఒక స్థానంలో ఆగుతుంది. కానీ $r = 2$ దగ్గర రెండు చెల్లుబాటు subarrays ఉన్నాయి: $[3, 3]$ మరియు $[3]$.

"సరిగ్గా K " అనేది ఒక కిటికీ కాదు – అది రెండు కిటికీల మధ్య ఉన్న జాగా.

10. Step – తేడాగా రాయడం

```
const exactlyKDistinct = (a, K) => atMostKDistinct(a, K) - atMostKDistinct(a, K - 1);
```

ఒక్క పంక్తి, మరియు అది ఎందుకు సరైనదో ఒక్క వాక్యం

"గరిష్టంగా K " లో సరిగ్గా $0, 1, 2 \dots K$ ఉన్నవన్నీ ఉన్నాయి.

"గరిష్టంగా $K-1$ " లో సరిగ్గా $0, 1, 2 \dots K-1$ ఉన్నాయి.

తేడా = సరిగ్గా K . ముగిసింది.

ఈ ఉపాయం సాధారణమైనది: "సరిగ్గా X " కి నేరుగా sliding window పనిచేయనప్పుడు, "గరిష్టంగా X " రాసి తేడా తీసుకోండి – ఎందుకంటే "గరిష్టంగా" ఏకదిశగా ఉంటుంది, "సరిగ్గా" ఉండదు.

మరియు ఇది §7 యొక్క అదే పరీక్ష: " r కదిలితే వేర్వేరు మూలకాల సంఖ్య పెరుగుతుంది మాత్రమే" ✓ – కానీ "సరిగ్గా K " కి ఆ వాక్యం అర్థవంతం కాదు, ఎందుకంటే పెరగడం వల్ల షరతు నిజం నుంచి అబద్ధం అవుతుంది, ఆపై మళ్ళీ కాదు.

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code • 1.2 లక్షల cases • mutation testing

ఏడు నియమాలు, అన్నీ **brute force** తో పోల్చినవి: **longestSumAtMost = brute force**, జవాబు $0..n$ పరిధిలో, **atMostKDistinct = brute force**, **exactlyK = brute force**, **atMost K** పెరిగితే తగ్గకూడదు, **atMost(n) = $n(n+1)/2$** , **twoSumSorted** నిజంగా ఒక జతని చూపాలి.

400 runs • 1,20,000 cases

నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

cases	1,20,000
empty	13,411
allSame	14,893
exactPos	65,339

==== mutation testing ====

మార్పు లేని code	→ 0/400 విఫలం
కీటికీ కుంచించడం `while` కి బదులు `if`	→ 0/150 విఫలం
atMost lo కుంచించడం `while` కి బదులు `if`	→ 150/150 విఫలం
ఉదా: atMost([4,4,1,3,2,1,4,0], 3) = 26, ఆశించినది 23	
`s > k` ని `s >= k` గా	→ 150/150 విఫలం
ఉదా: longest([4,4,1,3,2,1,4,0], 11) = 5, ఆశించినది 6	
atMost lo `c += r - l + 1` ని `c += 1` గా	→ 150/150 విఫలం
ఉదా: atMost([0,4,8,8,2], 3) = 5, ఆశించినది 14	
`m.size > K` ని `m.size >= K` గా	→ 150/150 విఫలం
ఉదా: atMost([0,4,8,8,2], 3) = 11, ఆశించినది 14	
exactlyK ని atMost(K) గా (తేడా తీసుకోకుండా)	→ 150/150 విఫలం
ఉదా: exactly([0,4,8,8,2], 3) = 14, ఆశించినది 3	
Map నుంచి సున్నా లెక్క ఉన్న key ని తొలగించకపోతే	→ 150/150 విఫలం
ఉదా: atMost అనంత వలయం	
twoSum lo `i < j` ని `i <= j` గా	→ 150/150 విఫలం
ఉదా: twoSum([1,2,4,6,6,8], 4) = [1,1] తప్పు	

ఏడు మార్పుల్లో ఏడూ పట్టుబడ్డాయి. మరియు మొదటి వరుస - **while** కి బదులు **if** - బతికింది. అది ఈ doc lo అత్యంత ఆసక్తికరమైన ఫలితం.

12. ఒకే bug, రెండు వేర్వేరు ఫలితాలు

`while (s > k)` ని `if (s > k)` గా మార్చడం – అంటే ఒక్క అడుగులో **ఒక్కసారే** కుంచించడం. అది స్పష్టంగా తప్పులా అనిపిస్తుంది. కానీ fuzz **0/150** చూపించింది.

కాబట్టి దాన్నే వదిలేయకుండా, **ప్రత్యేకంగా వెతికాను** – పెద్ద విలువలతో, ఒక అడుగులో చాలాసార్లు కుంచించాల్సిన పరిస్థితులతో:

```
పెద్ద విలువలతో (0..39, k ≤ 24) · 5,00,000 cases
while (సరైనది) → తప్పు: 0
if (mutation) → తప్పు: 0 (0.00%)
```

ఇంకా సున్నా. అప్పుడు అదే మార్పుని లెక్కపెట్టే ప్రశ్న మీద పరీక్షించాను:

అదే `while`→`if` మార్పు, రెండు వేర్వేరు ప్రశ్నల మీద · 2,00,000 cases

ప్రశ్న	while if
పొడవైన subarray (గరిష్ఠం)	0 0
ఎన్ని subarrays (లెక్క)	0 43379 (21.7%)

ఉదా: a = [3,4,3,0,5,1] · K = 3 → if 18, నిజం 16

ఒకే మార్పు · గరిష్ఠం ప్రశ్నలో 0% తప్పు · లెక్క ప్రశ్నలో 21.7%

కారణం: **గరిష్ఠం** అడిగినప్పుడు, కిటికీ చెల్లనిదైతే మనం దాన్ని **లెక్కలోకి తీసుకోము** (`if (s <= k)` అనే కాపలా). కాబట్టి 1 కొంచెం వెనకబడినా, తర్వాతి అడుగుల్లో అది పట్టుకుంటుంది, మరియు **best** ఎప్పుడూ **చెల్లుబాటు అయ్యే** కిటికీల నుంచే వస్తుంది.

లెక్కపెట్టటప్పుడు ఆ కాపలా లేదు – `c += r - l + 1` అనేది `l` సరైన స్థానంలో ఉందని **నమ్ముతుంది**. వెనకబడిన `l` అంటే కిటికీ చాలా పెద్దది, అంటే **ఎక్కువ లెక్క**.

ఇక్కడి పాఠం – MUTATION TESTING గురించి

ఒక mutation బతికితే మూడు అవకాశాలు (DSA Deep 01 lo చూశాం): బలహీనమైన పరీక్ష చచ్చిన code, లేదా harness సృష్టించలేని పరిస్థితి.

ఇక్కడ **నాలుగోది**: ఆ పంక్తి **ఒక వాడుకలో నిజంగా అవసరం లేదు**, ఇంకొక వాడుకలో తప్పనిసరి.

కాబట్టి "ఈ code సరైనదా?" అనేది పూర్తి ప్రశ్న కాదు. పూర్తి ప్రశ్న: **"ఏ ప్రశ్నకి సరైనది?"** – మరియు అదే `while` రాయడానికి కారణం: అది **రెండింటికీ** సరైనది.

13. Interview lo ఎలా రాయాలి

అడుగు	ఏమి చేయాలి
1	ముఖాత్మకాలు ఉన్నాయా అని అడగండి. జవాబు "ఉన్నాయి" అయితే sliding window వాడొద్దు
2	§7 యొక్క వ్యాఖ్యని బిగ్గరగా చెప్పండి – "r కదిలితే కొలత పెరుగుతుంది మాత్రమే, ఎందుకంటే.."
3	కుంచించడానికి ఎప్పుడూ while, if కాదు – §12
4	లెక్కపెడుతుంటే <code>c += r - l + 1</code> , <code>c++</code> కాదు
5	"సరిగ్గా K" కనిపిస్తే వెంటనే <code>atMost(K) - atMost(K-1)</code>

14. Follow-ups – గరిష్ట కిటికీ, అక్షరాలు, prefix sums

"ముఖాత్మకాలు ఉన్నప్పుడు $\text{sum} = k$ సరిగ్గా ఉన్న subarrays?"

Sliding window కాదు – **prefix sums + hash map**. $P[j] - P[i] = k$ అంటే $P[i] = P[j] - k$. కాబట్టి ఎడమ నుంచి కుడికి నడుస్తూ ప్రతి $P[j]$ దగ్గర $P[j] - k$ ఇప్పటిదాకా ఎన్నిసార్లు కనిపించిందో hash map lo చూడటం. $O(n)$, మరియు **ముఖాత్మకాలతో పనిచేస్తుంది** – ఎందుకంటే ఇది కిటికీ గురించి కాదు, రెండు బిందువుల గురించి.

"కిటికీలో గరిష్ట విలువ కావాలంటే?"

`Math.max(...window)` ప్రతి అడుగుకీ $O(k)$ – మొత్తం $O(nk)$. దానికి బదులు **monotonic deque**: deque lo సూచికలు తగ్గుతున్న క్రమంలో ఉంచడం, ముందు నుంచి గడువు ముగిసినవి తీసేయడం. ప్రతి మూలకం deque lo ఒకసారే చేరి ఒకసారే పోతుంది $\rightarrow O(n)$. ఇది sliding window యొక్క అదే వాదన, వేరే నిర్మాణం మీద.

"అక్షరాల problems (anagram, longest substring)?"

అదే pattern, Map కి బదులు 26-పొడవు array (లేదా Unicode అయితే Map). §7 యొక్క వాక్యం ఇక్కడా వర్తిస్తుంది: "r కదిలితే ఒక అక్షరం లెక్క పెరుగుతుంది మాత్రమే" ✓.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **monotonic deque** (పైన వివరించాను, fuzz చేయలేదు), **prefix-sum + hash map** విధానం, మరియు **స్థిర పొడవు కిటికీలు** – అవి ఇంకా సులభం (కుంచించే loop లేదు), కాబట్టి ఇక్కడి విరుపులు వాటికి వర్తించవు.

15. ఏమి నేర్చుకున్నాం

1. ఈ pattern యొక్క వేగం ఒక ఊహా మీద ఆధారపడి ఉంది. "l వెనక్కి వెళ్ళినవసరం లేదు" – అదే $O(n)$ ఇచ్చింది, మరియు అదే తప్పు జవాబు ఇచ్చింది.
2. ఒక్క ఋణాత్మక సంఖ్య చాలు. $0..9 \rightarrow 0\%$ తప్పు. $-1..9 \rightarrow 1.77\%$. $-5..9 \rightarrow 14.77\%$. సున్నా సరే – ఇది దిశ గురించి, పరిమాణం గురించి కాదు.
3. "సరిగ్గా K" ఒక కిటికీ కాదు. నేరుగా లెక్కపెడితే **47.8% తప్పు**. సరైనది ఒక తేడా: `atMost(K) – atMost(K-1)`.
4. లెక్కపెట్టేటప్పుడు ఒక్కొక్కటిగా కాదు. `c += r - l + 1` – ఎందుకంటే చెల్లుబాటు కిటికీ యొక్క ప్రతి suffix కూడా చెల్లుతుంది.
5. ఒక bug, రెండు వేర్వేరు ఫలితాలు. `while` $\rightarrow$ `if` గరిష్ఠం ప్రశ్నలో **5 లక్షల cases lo 0 తప్పు**, లెక్క ప్రశ్నలో **21.7%**.
6. అందుకే "ఈ code సరైనదా?" అనేది అసంపూర్ణ ప్రశ్న. సరైన ప్రశ్న: "ఏ ప్రశ్నకి సరైనది?"
7. Binary search యొక్క అదే పాఠం, వేరే రూపంలో. అక్కడ predicate ఏకదిశగా ఉండాలి. ఇక్కడ కొలత ఏకదిశగా మారాలి. రెండూ ఒకే ప్రశ్న.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Sliding window ఒక ఉపాయం కాదు – అది ఒక ఒప్పందం.

మీరు ఇస్తున్నది: "నేను ఎడమ pointer ని ఎప్పుడూ వెనక్కి తీసుకురాను."
దానికి బదులుగా పొందేది: $O(n^2)$ కి బదులు $O(n)$.

ఆ ఒప్పందం చెల్లాలంటే **ఒక్క షరతు** నిజం కావాలి – కిటికీ పెరిగితే కొలత ఒకే దిశలో కదలాలి. ఆ షరతు నిజమో కాదో **code చెప్పదు, crash కాదు, మీ మూడు ఉదాహరణలు నెగ్గుతాయి.**

కాబట్టి ఆ షరతుని మీరే, ప్రతిసారీ, ఒక వాక్యంలో రాయాలి.